

Backpressure Patterns: Staying Alive Under Load



Published on August 7, 2022



Webstack
Builders

Table of Contents

Understanding Overload	3
The Anatomy of Cascading Failure	3
Little's Law and Queue Theory	5
Goodput vs. Throughput	5
Backpressure Mechanisms	6
What Is Backpressure?	6
Explicit vs. Implicit Backpressure	7
TCP Flow Control as Inspiration	8
Admission Control	9
The Bouncer Pattern	9
Admission Control Strategies	10
Priority-Based Admission	12
Load Shedding	13
What to Shed and When	13
LIFO Shedding: Drop the Newest	14
CoDel: Controlled Delay	15
Graceful Degradation Tiers	17
Client Communication	20
HTTP Status Codes for Overload	20
Retry-After Header	20
Client-Side Backoff Patterns	22
Implementation Patterns	25
Circuit Breakers for Downstream Protection	25
Bulkheads for Isolation	27
Adaptive Concurrency Limits	29
Testing Overload Handling	32
Load Testing for Backpressure	32
Chaos Engineering for Overload	34
Observability for Overload	36
Key Metrics to Monitor	36
Dashboard Layout	38
Conclusion	40



Most systems don't fail because they can't handle load. They fail because they try to handle *all* the load. Without explicit overload handling, a system accepts work it cannot complete, queues grow unbounded, latency spikes, timeouts cascade, and what could have been a recoverable traffic spike becomes a prolonged outage.

I watched this happen to an e-commerce platform during a flash sale. Traffic spiked to 10x normal within minutes. The system accepted every request – that was the problem. Database connections exhausted. Response times climbed from 200ms to 30 seconds. Clients started timing out and retrying, which doubled the load again. The operations team tried scaling up, but new instances couldn't get database connections either. What could have been a 15-minute degradation became a 4-hour outage that required a full restart to clear the request backlog.

The fix wasn't more capacity. It was teaching the system to say "no."

Backpressure is a survival mechanism. It's the ability to signal upstream that you're overwhelmed and cannot accept more work right now. Done well, it keeps your system responsive at capacity while gracefully degrading beyond it. Done poorly – or not at all – your system accepts work until it collapses.

WARNING

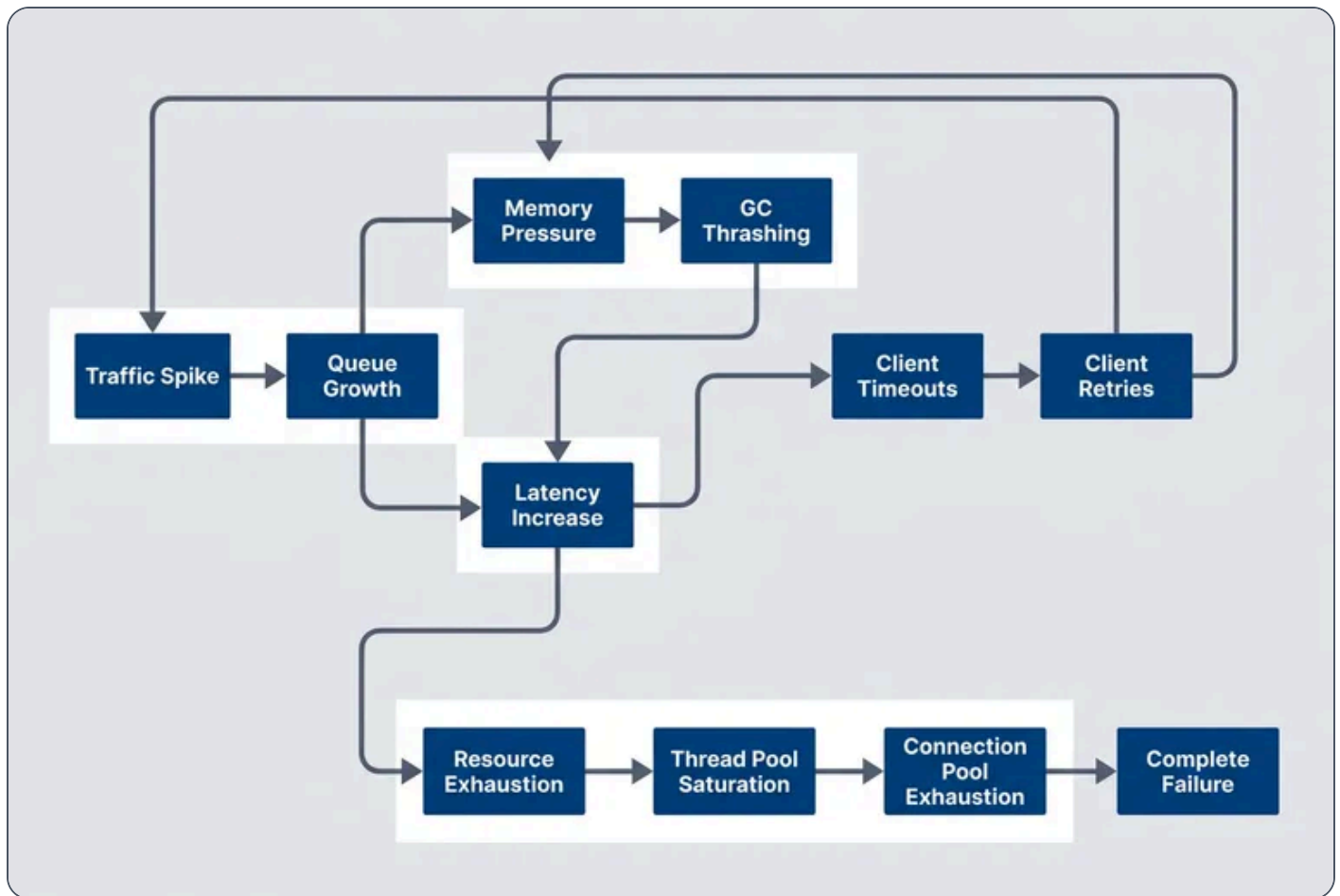
A system without backpressure will accept work until it collapses. The goal is not to handle unlimited load – it's to handle your design capacity reliably and degrade gracefully beyond it.

Understanding Overload

The Anatomy of Cascading Failure

Overload doesn't just make systems slow – it makes them *worse*. There's a vicious cycle at work: load increases, latency increases, clients timeout, clients retry, and now you have even more load. What started as a 2x traffic spike becomes a 4x spike from retries alone.





The cascading failure loop where overload compounds itself through retries and resource exhaustion

The diagram shows three paths to failure, and they often happen simultaneously. The retry loop is the most insidious – well-intentioned client retry logic turns a temporary overload into a sustained assault. Thread pool saturation means requests that *could* be processed sit waiting for a worker. And memory pressure from growing queues triggers garbage collection pauses that reduce your effective capacity right when you need it most.

This is why overload tends to get worse rather than self-correcting. Every mechanism designed for reliability under normal conditions – retries, timeouts, connection pooling – becomes a liability under overload.



 INFO

When you don't need this: If your system can scale horizontally fast enough to absorb traffic spikes (autoscaling in under 30 seconds, stateless services), you may not need sophisticated backpressure. But most systems have *something* that doesn't scale – a database, a third-party API, a license-limited service. That's where these patterns become essential.

Little's Law and Queue Theory

There's a fundamental relationship between throughput, latency, and queue depth that explains why overload causes latency to explode:

$$L = \lambda \times W$$

Where L is the average number of items in the system (queue depth), λ is the arrival rate (requests per second), and W is the average time in system (latency).

This equation is deceptively simple but has profound implications:

- 1 Example: System processing 100 RPS with 200ms latency
- 2 Queue depth = $100 \times 0.2 = 20$ requests in flight
- 3
- 4 Same system under overload: 500 RPS arriving, still processing 100 RPS
- 5 Effective latency grows as queue builds
- 6 After 1 minute: 24,000 requests queued, latency > 4 minutes

When arrival rate exceeds processing rate, the queue grows without bound. And here's the cruel part: every request in the queue adds to *everyone's* wait time. A request that arrives when 1,000 others are queued will wait for all 1,000 to be processed first. By the time it's served, the client has likely given up.



 INFO

Little's Law explains why overload causes latency to explode. When arrival rate exceeds processing rate, queue depth grows without bound, and every queued request adds to everyone's wait time.

Goodput vs. Throughput

Here's a counterintuitive truth: throughput can *increase* during overload while your system becomes effectively useless. The distinction is between throughput (work attempted) and goodput (work successfully completed).

Metric	Definition	Under Normal Load	Under Overload
Goodput	Successful responses	1000 RPS	400 RPS
Throughput	Requests processed	1000 RPS	1200 RPS
Badput	Failed/timed out	0 RPS	800 RPS

Throughput can increase during overload while goodput collapses.

Under overload, your system might process more requests than ever – but most of that work is wasted. Requests timeout after consuming resources. Database queries complete but the client already gave up. Retries succeed but the original request already errored out.

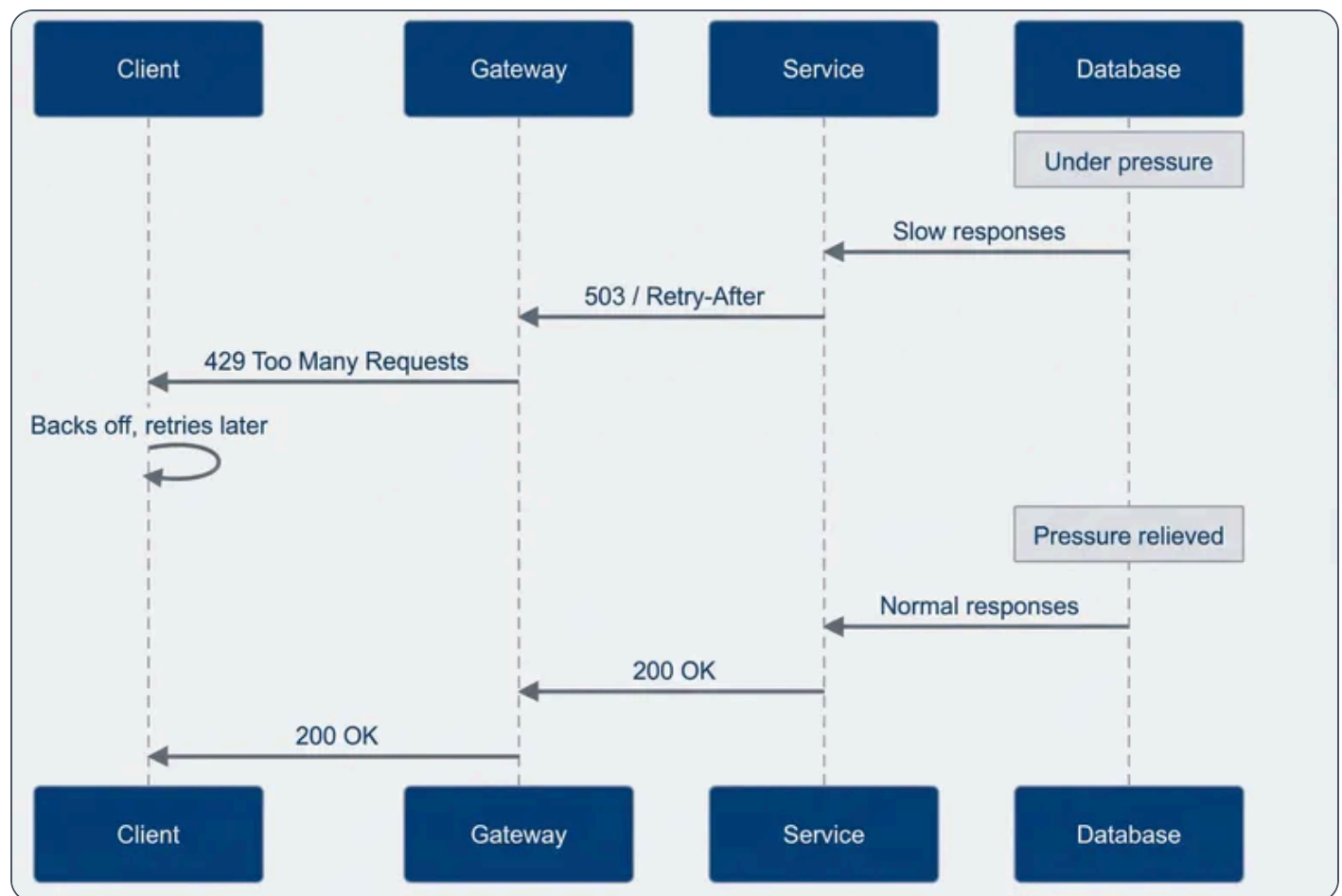
This is the key insight: if you'd simply rejected 1,000 of those 2,000 requests immediately, you could have successfully served the other 1,000. Instead, by accepting all of them, you served only 400. Fast rejection is better than slow failure.



Backpressure Mechanisms

What Is Backpressure?

Backpressure is the propagation of flow control signals from consumers back to producers. When a downstream component is overwhelmed, it signals upstream to slow down. The signal travels backward through the system until it reaches something that can actually reduce the load – usually the client.



Backpressure propagates from the constrained resource back to the client

The key principle I keep coming back to: you can't process your way out of overload. The only thing that actually reduces load is getting clients to send fewer requests. Backpressure is the mechanism that makes that happen.



Explicit vs. Implicit Backpressure

I've seen several ways to signal overload, each with different tradeoffs:

#	Type	Mechanism	Example	Pros	Cons
1	Explicit	Protocol-level signals	HTTP 429, TCP flow control	Clear signal, client can react	Requires client cooperation
2	Implicit	Increased latency	Slow responses	No implementation needed	Clients may not notice, retry
3	Connection-based	Refuse connections	Connection limits	Hard boundary	Poor client experience
4	Queue-based	Bounded queues	Reject when full	Predictable latency	Work is lost

Backpressure signaling techniques with pros/cons.

Explicit backpressure is almost always better than implicit. When you return a slow response, clients don't know if you're overloaded or just processing a complex request. They might retry, making things worse. When you return a 429 with a Retry-After header, the signal is unambiguous: "I'm busy, come back in 30 seconds."

TCP Flow Control as Inspiration

TCP solved the backpressure problem decades ago. Its flow control mechanism provides a model worth studying:

TCP Flow Control:

1

Receiver advertises window size:

It tells the sender how much data it can accept.



2

Sender limits in-flight data:

It caps in-flight data at the advertised window size.

3

As receiver processes data:

The available window opens again.

4

If receiver is slow:

The window can shrink to zero, which creates backpressure.

Application equivalent:

1

Service advertises capacity:

It communicates capacity through rate limits and queue depth.

2

Clients limit concurrent requests:

They keep concurrency within that advertised capacity.

3

As service recovers:

Available capacity opens back up.

4

If service is slow:

Clients back off and send less work.

The elegance of TCP's approach is that backpressure is *automatic*. The receiver doesn't have to decide when to apply pressure – the window size naturally reflects its processing capacity. Application-level backpressure often requires more explicit coordination, but the principle is the same: consumers must be able to tell producers “slow down.”

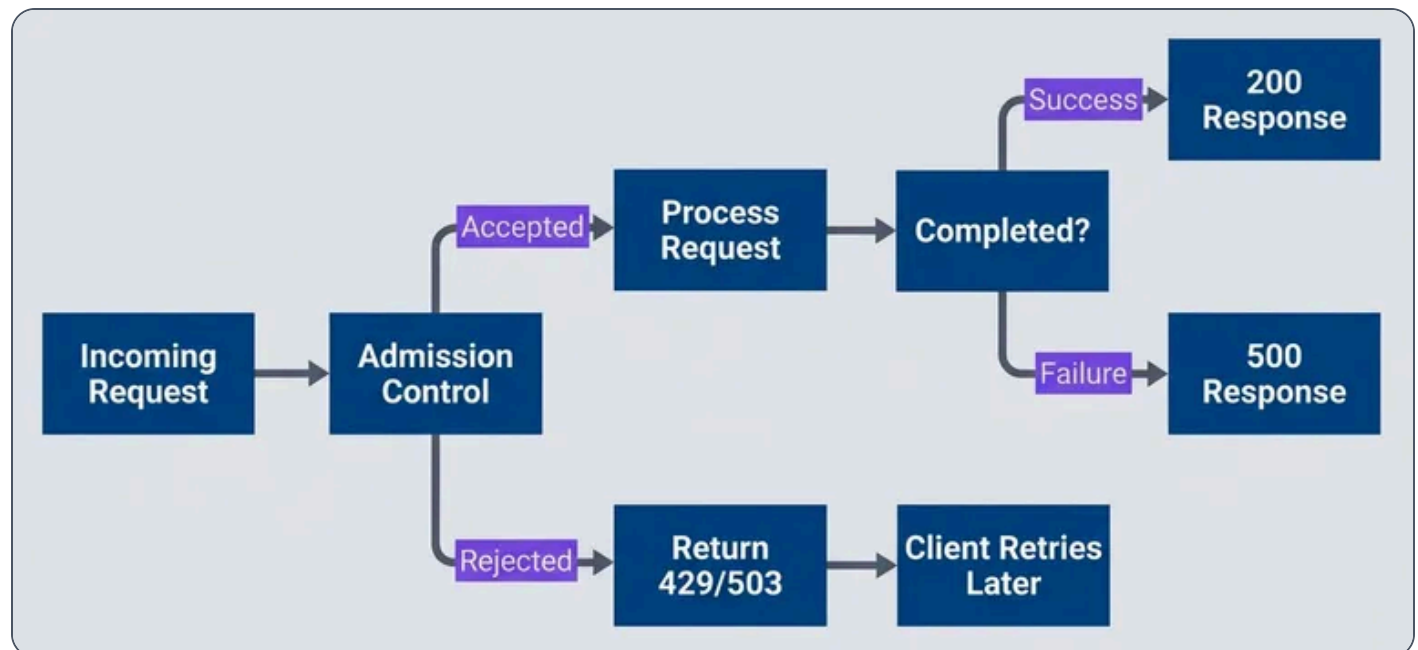


Admission Control

The Bouncer Pattern

Think of admission control like a nightclub bouncer. The bouncer's job isn't to throw people out after they're inside causing trouble – it's to decide at the door who gets in. If the club is at capacity, new arrivals wait outside. The people inside have a good experience because it's not overcrowded.

Admission control works the same way: decide *upfront* whether to accept work, rather than accepting everything and failing later. A request rejected at the door costs almost nothing. A request that gets halfway through your system before timing out has consumed database connections, CPU cycles, and memory – resources that could have served requests you *could* complete.



Admission control rejects work at the door rather than accepting and failing

✓ SUCCESS

It's better to reject 20% of requests immediately than to accept 100% and fail 40% after wasting resources on them. Fast rejection preserves capacity for requests you can actually serve.



Admission Control Strategies

I've found several approaches to deciding which requests to admit, each with different tradeoffs:

```
1  interface AdmissionController {
2      shouldAdmit(request: Request): boolean;
3  }
4
5  // Strategy 1: Simple rate limiting
6  // Allows N requests per second, rejects the rest
7  class RateLimitAdmission implements AdmissionController {
8      private tokens: number;
9      private readonly maxTokens: number;
10     private readonly refillRate: number;
11
12     shouldAdmit(request: Request): boolean {
13         if (this.tokens > 0) {
14             this.tokens--;
15             return true;
16         }
17         return false;
18     }
19 }
20
21 // Strategy 2: Concurrency limiting
22 // Allows N requests in-flight simultaneously
23 class ConcurrencyAdmission implements AdmissionController {
24     private inFlight: number = 0;
25     private readonly maxConcurrent: number;
26
27     shouldAdmit(request: Request): boolean {
28         if (this.inFlight < this.maxConcurrent) {
29             this.inFlight++;
30             return true;
31         }
32         return false;
33     }
34 }
35
36 // Strategy 3: Latency-based (adaptive)
37 // Rejects more aggressively as latency increases
```



```

38 class LatencyAdmission implements AdmissionController {
39     private recentLatencies: number[] = [];
40     private readonly targetLatency: number;
41
42     shouldAdmit(request: Request): boolean {
43         const p99 = this.calculateP99();
44         const admitProbability = this.targetLatency / Math.max(p99, this.targetLatency);
45         return Math.random() < admitProbability;
46     }
47 }
48
49 // Strategy 4: Queue depth based
50 // Rejects when queue exceeds threshold
51 class QueueDepthAdmission implements AdmissionController {
52     private queueDepth: number = 0;
53     private readonly maxQueueDepth: number;
54
55     shouldAdmit(request: Request): boolean {
56         return this.queueDepth < this.maxQueueDepth;
57     }
58 }

```

Rate limiting is the simplest – it caps throughput regardless of what the system can actually handle. Concurrency limiting is better because it naturally adapts: if requests are fast, you can serve more per second; if they’re slow, you serve fewer but don’t overload. Latency-based admission is the most sophisticated – it directly targets the metric you care about (response time) and adjusts admission probability based on observed behavior.

Priority-Based Admission

Not all requests are equal. During overload, you want to protect critical traffic – health checks, payment processing, authenticated users – while shedding less important work. Priority-based admission reserves capacity for high-priority requests.

aws-alb-priority-rules.yaml

```

1 # AWS ALB listener rules with priority-based routing
2 # Higher priority numbers = lower precedence
3 Rules:

```



```
4   - Priority: 1
5     Conditions:
6       - Field: path-pattern
7         Values: ["/health", "/api/payments/*"]
8     Actions:
9       - Type: forward
10        TargetGroupArn: !Ref CriticalTargetGroup
11
12  - Priority: 10
13    Conditions:
14      - Field: http-header
15        HttpHeaderConfig:
16          HttpHeaderName: Authorization
17          Values: ["Bearer *"]
18    Actions:
19      - Type: forward
20        TargetGroupArn: !Ref AuthenticatedTargetGroup
21
22  - Priority: 100
23    Conditions:
24      - Field: path-pattern
25        Values: ["/*"]
26    Actions:
27      - Type: forward
28        TargetGroupArn: !Ref AnonymousTargetGroup
```

Priority-based routing configuration for AWS Application Load Balancer.

The key insight is that critical traffic can *borrow* capacity from lower tiers. If the anonymous quota is exhausted but the critical quota has room, anonymous requests are rejected while critical requests continue flowing. During severe overload, you shed in reverse priority order: anonymous first, then authenticated, and critical traffic last (or never).

Load Shedding

What to Shed and When

Load shedding is the deliberate dropping of work to preserve system stability. It sounds brutal – and it is – but it's also essential. When you're drowning, you throw cargo overboard to keep the ship afloat. The alternative is losing everything.



The key insight is that *some* work completing successfully is better than *all* work failing. If your system can handle 1,000 RPS (Requests Per Second) but 2,000 are arriving, you have a choice: accept all 2,000 and watch goodput collapse to 400 (because everything times out), or shed 1,000 immediately and successfully serve the other 1,000.

#	Shedding Strategy	When to Use	Tradeoff
1	Random	Simple implementation	May drop important requests
2	LIFO (newest first)	Reduce queuing latency	May serve stale requests
3	FIFO (oldest first)	Likely already timed out	Wasted work on old requests
4	Priority-based	Protect important traffic	Complexity, starvation risk
5	Cost-based	Protect expensive operations	Requires cost tracking

Load shedding strategies and their tradeoffs.

The choice of strategy depends on your workload. Random shedding is the simplest – it’s statistically fair and requires no tracking. But “fair” isn’t always optimal. Sometimes you want to be deliberately unfair to maximize successful completions.

LIFO Shedding: Drop the Newest

Here’s a counterintuitive idea: when you need to shed load, drop the *newest* requests first, not the oldest. Traditional queuing is FIFO (First In, First Out) – first in, first out. LIFO (Last In, First Out) shedding inverts this.

Consider a queue with 100 requests where processing takes 100ms each:

FIFO (First In, First Out) shedding (traditional):

Request 1 (oldest): | It waited 10 seconds and will still be served.

Request 100 (newest): | It will wait more than 10 seconds and will likely time out.



Outcome:	The client likely already gave up on request 1, so the work was wasted.
-----------------	---

LIFO (Last In, First Out) shedding (counterintuitive but effective):

Request 100 (newest):	It is rejected immediately, so the client can retry.
------------------------------	--

Request 1 (oldest):	It is served, but the client may already have given up.
----------------------------	---

Better:	A CoDel-style approach considers wait time before dropping work.
----------------	--

The logic is this: a request that just arrived and gets rejected immediately can be retried right away. The client is still waiting, the context is still fresh. A request that waited 30 seconds in a queue and finally gets served? That client probably gave up long ago. The response goes nowhere. You did all that work for nothing.

INFO

LIFO (Last In, First Out) shedding sounds unfair but often produces better outcomes. A request that just arrived and gets rejected can be retried immediately. A request that waited 30 seconds and finally gets served may find the client already gave up.

CoDel: Controlled Delay

Neither pure FIFO (First In, First Out) nor LIFO (Last In, First Out) is ideal. What we really want is to drop requests that have *already waited too long*—requests whose clients have likely given up. This is exactly what CoDel (Controlled Delay) (Controlled Delay) does.

CoDel (Controlled Delay) was originally designed for network routers, but its principles apply perfectly to application queues. The algorithm tracks “sojourn time”—how long each item has been in the queue. If sojourn time stays below a target (say, 5ms), the queue is healthy. If it stays *above* the target for an entire interval (say, 100ms), CoDel (Controlled Delay) starts dropping.

The clever part is *how* it drops. Rather than dropping everything, CoDel (Controlled Delay) drops at an accelerating rate proportional to the square root of how many consecutive drops have occurred. This creates just enough backpressure to bring the queue under control without overreacting.



codel-queue.go

```

1  // CoDel queue for use as a WASM filter in Envoy – either as
2  // reverse proxy/API gateway middleware or service mesh sidecar.
3
4  package codel
5
6  import (
7      "math"
8      "time"
9  )
10
11 type CoDelConfig struct {
12     TargetDelay time.Duration // Target queuing delay (e.g., 5ms)
13     Interval    time.Duration // Observation interval (e.g., 100ms)
14 }
15
16 type entry[T any] struct {
17     item          T
18     enqueueTime time.Time
19 }
20
21 type CoDelQueue[T any] struct {
22     config      CoDelConfig
23     queue       []entry[T]
24     dropping    bool
25     firstAboveTime time.Time
26     dropNext    time.Time
27     count       int
28 }
29
30 func (q *CoDelQueue[T]) Enqueue(item T) {
31     q.queue = append(q.queue, entry[T]{
32         item:          item,
33         enqueueTime: time.Now(),
34     })
35 }
36
37 func (q *CoDelQueue[T]) Dequeue() (T, bool) {
38     var zero T
39     if len(q.queue) == 0 {
40         return zero, false
41     }
42

```



```
43     e := q.queue[0]
44     q.queue = q.queue[1:]
45     sojournTime := time.Since(e.enqueueTime)
46
47     if sojournTime < q.config.TargetDelay {
48         // Queue delay is acceptable
49         q.firstAboveTime = time.Time{}
50         q.dropping = false
51         return e.item, true
52     }
53
54     // Queue delay exceeds target
55     if q.firstAboveTime.IsZero() {
56         q.firstAboveTime = time.Now()
57     } else if time.Since(q.firstAboveTime) > q.config.Interval {
58         // Been above target for too long, start dropping
59         q.dropping = true
60         q.count++
61         q.dropNext = time.Now().Add(
62             time.Duration(float64(q.config.Interval) / math.Sqrt(float64(q.count))),
63         )
64     }
65
66     if q.dropping && time.Now().After(q.dropNext) {
67         // Drop this item and schedule next drop
68         q.count++
69         q.dropNext = time.Now().Add(
70             time.Duration(float64(q.config.Interval) / math.Sqrt(float64(q.count))),
71         )
72         return zero, false // Dropped
73     }
74
75     return e.item, true
76 }
```

CoDel (Controlled Delay) (Controlled Delay) queue implementation that drops based on sojourn time.

The beauty of CoDel (Controlled Delay) is that it's self-tuning. During normal operation, it never drops anything. When overload hits, it drops just enough to keep latency bounded. When overload subsides, it automatically stops dropping. No manual tuning of drop rates required.



Graceful Degradation Tiers

Load shedding doesn't have to be all-or-nothing. A smarter approach is *graceful degradation*: as load increases, progressively disable non-essential features to preserve capacity for what matters.

Think of it like a submarine diving deeper. At each depth threshold, you seal off another compartment. The ship loses functionality but gains survivability. At maximum depth, only the core systems remain operational – but the ship is still alive.

graceful-degradation-tiers.yaml

```
1  # Feature flags for use with LaunchDarkly, Unleash, Flagsmith, or similar.
2  # Application code checks load metrics against thresholds at runtime
3  # and toggles features on / off accordingly to conserve resources.
4  degradation_tiers:
5    normal:
6      trigger: "load < 80%"
7      recover_at: "load < 80%" # No hysteresis at baseline
8      features_enabled:
9        - full_search_results
10       - personalized_recommendations
11       - real_time_analytics
12       - detailed_error_messages
13
14    elevated:
15      trigger: "load >= 80%"
16      recover_at: "load < 70%" # Hysteresis: enter at 80%, exit at 70%
17      features_disabled:
18        - personalized_recommendations
19      features_degraded:
20        - search_results: 10 # Reduced from default
21      features_enabled:
22        - basic_search
23        - cached_recommendations
24        - delayed_analytics
25
26    critical:
27      trigger: "load >= 90%"
28      recover_at: "load < 80%" # Hysteresis
29      features_disabled:
```

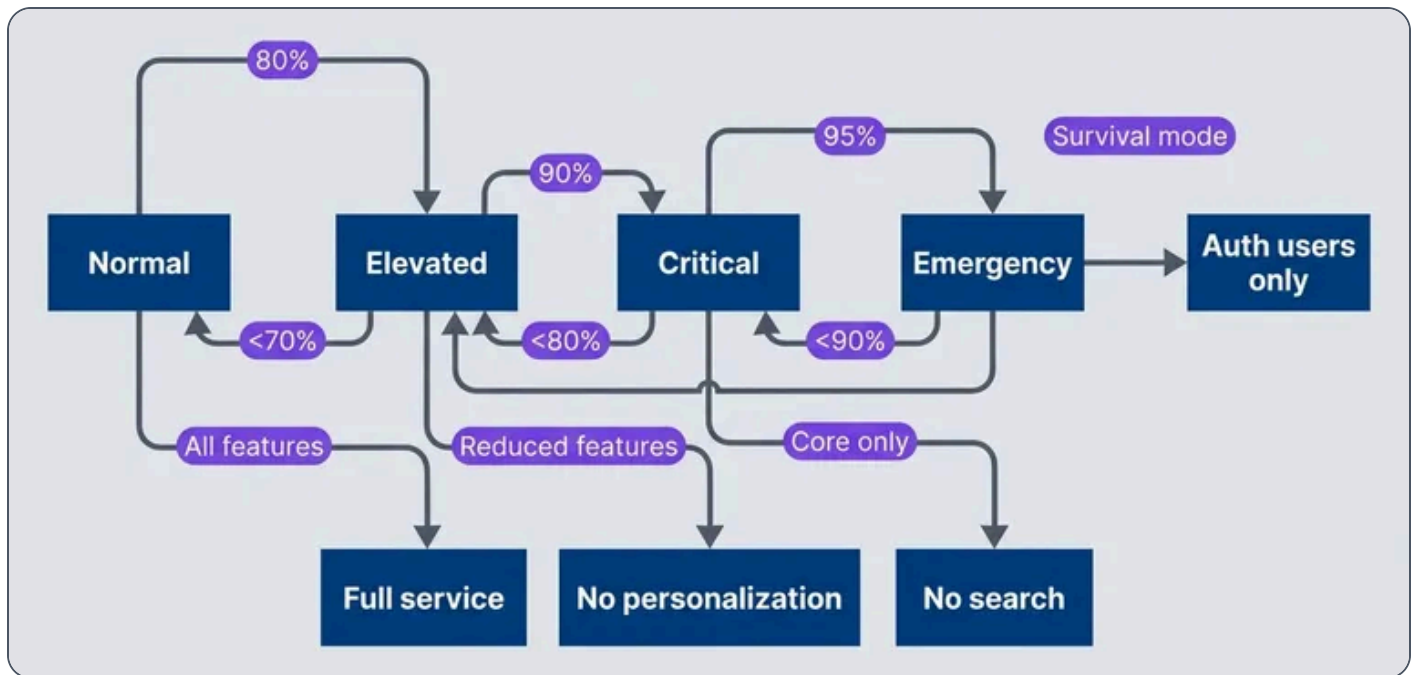


```
30     - search
31     - analytics
32     - personalized_recommendations
33     features_enabled:
34     - core_transactions_only
35     - static_content
36     - essential_apis_only
37
38     emergency:
39     trigger: "load >= 95%"
40     recover_at: "load < 90%" # Hysteresis
41     features_disabled:
42     - anonymous_traffic
43     - non_critical_apis
44     features_enabled:
45     - authenticated_users_only
46     - payment_processing
47     - health_endpoints
```

Multi-tier graceful degradation configuration.

Notice the *hysteresis* in the diagram below – the thresholds to recover are lower than the thresholds to degrade. You enter “elevated” at 80% but don’t return to “normal” until you’re below 70%. This prevents oscillation. Without hysteresis, a system hovering around 80% would constantly flip between normal and elevated, which is worse than staying in one state.





Degradation tiers with hysteresis to prevent oscillation

✓ SUCCESS

Graceful degradation requires knowing which features are expendable. Work with product owners to classify features into tiers *before* an incident. During an outage is the wrong time to debate whether search is more important than recommendations.

Client Communication

HTTP Status Codes for Overload

Backpressure only works if clients understand the signal. HTTP provides specific status codes for communicating overload – use them correctly, and well-behaved clients will back off automatically.



Status Code	Meaning	When to Use	Client Action
429	Too Many Requests	Rate limit exceeded	Back off, retry with delay
503	Service Unavailable	Temporary overload	Retry after delay
504	Gateway Timeout	Upstream timeout	May retry
507	Insufficient Storage	Queue/buffer full	Reduce request rate

HTTP status codes for overload and suggested client actions.

The distinction between 429 and 503 matters. A 429 says “you specifically are sending too many requests”—it’s a per-client signal. A 503 says “the service is overwhelmed regardless of who’s asking”—it’s a global signal. Rate limiters return 429; admission control during overload returns 503.

Avoid returning 500 for overload conditions. A 500 suggests a bug — something the client can’t fix by waiting. Clients may not retry 500s, or may retry them aggressively assuming it’s a transient glitch. Be explicit: if the problem is capacity, say so with 429 or 503.

Retry-After Header

The `Retry-After` header tells clients exactly how long to wait before trying again. Without it, clients are guessing — and they’ll usually guess too short, creating a thundering herd when they all retry simultaneously.

retry-after-middleware.ts

```

1  import { Request, Response, NextFunction } from 'express';
2
3  interface OverloadState {
4    isOverloaded: boolean;
5    estimatedRecoverySeconds: number;
6    currentLoad: number;
7  }
8
```



```
9  function getOverloadState(): OverloadState {
10    // Implementation that checks system metrics
11    return {
12      isOverloaded: true,
13      estimatedRecoverySeconds: 30,
14      currentLoad: 0.95,
15    };
16  }
17
18  function overloadMiddleware(req: Request, res: Response, next: NextFunction) {
19    const state = getOverloadState();
20
21    if (!state.isOverloaded) {
22      return next();
23    }
24
25    // Calculate retry delay based on load
26    const retryAfter = Math.min(
27      state.estimatedRecoverySeconds,
28      Math.ceil(state.currentLoad * 60) // Max 60 seconds
29    );
30
31    res.setHeader('Retry-After', retryAfter);
32    res.setHeader('X-RateLimit-Remaining', '0');
33    res.setHeader('X-RateLimit-Reset', Date.now() + retryAfter * 1000);
34
35    // Include machine-readable details
36    res.status(503).json({
37      error: 'service_unavailable',
38      message: 'Service is temporarily overloaded',
39      retry_after: retryAfter,
40      details: {
41        current_load: state.currentLoad,
42        estimated_recovery: `${retryAfter}s`,
43      },
44    });
45  }
```

Middleware that returns informative overload responses with Retry-After headers.

A few implementation notes: the `Retry-After` value can be either seconds (an integer) or an HTTP date. Seconds is simpler and avoids clock-sync issues. The `X-RateLimit-*` headers aren't standardized but are widely understood – include them for clients that check. Always include a JSON body with details; some



clients log these for debugging, and it makes your support team's life easier.

⚠ WARNING

Always include a Retry-After header when returning 429 or 503. Without it, clients have no guidance and may retry immediately, making the overload worse.

Client-Side Backoff Patterns

The server half of backpressure is useless without cooperative clients. A client that ignores 429s and retries immediately is actively making the problem worse. Good clients implement exponential backoff with jitter.

Exponential backoff means each retry waits longer than the last — typically doubling. After 5 retries with a 1-second base, you're waiting 32 seconds. This naturally spreads out retry storms. Jitter adds randomness to prevent synchronization — without it, 1,000 clients that all failed at the same time will all retry at the same time, recreating the original spike.

```
1  interface BackoffConfig {
2      initialDelayMs: number;
3      maxDelayMs: number;
4      multiplier: number;
5      jitter: number; // 0-1, percentage of randomization
6  }
7
8  class ExponentialBackoff {
9      private attempt: number = 0;
10
11     constructor(private config: BackoffConfig) {}
12
13     getNextDelay(retryAfterHeader?: string): number {
14         // Honor server's Retry-After if provided
15         if (retryAfterHeader) {
16             const serverDelay = parseInt(retryAfterHeader, 10) * 1000;
17             if (!isNaN(serverDelay)) {
18                 return this.addJitter(serverDelay);
19             }
20         }
21     }
22 }
```




```

20     }
21
22     // Calculate exponential backoff
23     const exponentialDelay = this.config.initialDelayMs *
24         Math.pow(this.config.multiplier, this.attempt);
25     const cappedDelay = Math.min(exponentialDelay, this.config.maxDelayMs);
26
27     this.attempt++;
28     return this.addJitter(cappedDelay);
29 }
30
31 private addJitter(delay: number): number {
32     const jitterRange = delay * this.config.jitter;
33     const jitter = (Math.random() - 0.5) * 2 * jitterRange;
34     return Math.max(0, delay + jitter);
35 }
36
37 reset(): void {
38     this.attempt = 0;
39 }
40 }
41
42 // Utility
43 const sleep = (ms: number) => new Promise(resolve => setTimeout(resolve, ms));
44
45 // Usage
46 const backoff = new ExponentialBackoff({
47     initialDelayMs: 1000,
48     maxDelayMs: 60000,
49     multiplier: 2,
50     jitter: 0.25,
51 });
52
53 async function fetchWithBackoff(url: string, maxRetries: number = 5) {
54     for (let i = 0; i < maxRetries; i++) {
55         try {
56             const response = await fetch(url);
57
58             if (response.status === 429 || response.status === 503) {
59                 const retryAfter = response.headers.get('Retry-After');
60                 const delay = backoff.getNextDelay(retryAfter ?? undefined);
61                 console.log(`Rate limited, waiting ${delay}ms before retry`);
62                 await sleep(delay);
63                 continue;

```



```
64     }
65
66     backoff.reset();
67     return response;
68   } catch (error) {
69     const delay = backoff.getNextDelay();
70     console.log(`Request failed, waiting ${delay}ms before retry`);
71     await sleep(delay);
72   }
73 }
74
75 throw new Error(`Failed after ${maxRetries} retries`);
76 }
```

The critical detail: always honor `Retry-After` when the server provides it. The server knows its load better than your exponential formula. If the server says “wait 30 seconds,” wait 30 seconds – don’t override with your calculated 2-second delay.

INFO

If you control both the client and server, you can do better than generic backoff. Have the server return current queue depth or estimated wait time, and let clients make informed decisions about whether to retry or fail fast to the user.

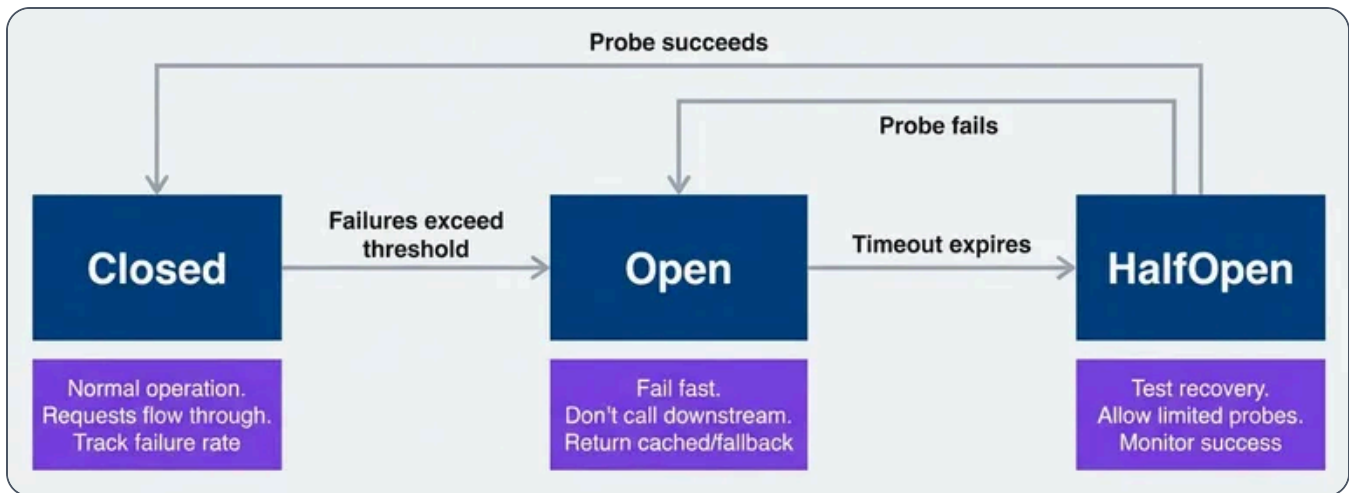
Implementation Patterns

Circuit Breakers for Downstream Protection

Circuit breakers prevent a failing downstream dependency from taking down your entire system. The pattern comes from electrical engineering: when current exceeds safe levels, the breaker trips and stops all current flow, protecting the circuit from damage.

In software, the “current” is requests to a downstream service. When that service starts failing, you don’t want to keep hammering it – that wastes your resources waiting for timeouts and makes the downstream service’s recovery harder. Instead, you “trip” the circuit: stop calling the failing service entirely, fail fast, and return a fallback or cached response.





Circuit breaker state machine for downstream protection

I think of the three states as a conversation between your service and its dependencies. **Closed** is normal operation where requests flow through and you track failure rates. When failures exceed a threshold, the breaker trips to **Open**—all requests fail immediately without calling downstream. After a timeout, it transitions to **Half-Open**, allowing a few probe requests through. If probes succeed, back to **Closed**; if they fail, back to **Open**.

```

1 // In application code. Gives fine-grained control per operation, not just
2 // per service. Allows custom fallback logic (return cached data, degraded
3 // response, etc.). Can protect non-network operations (database queries,
4 // file I/O). Popular libraries include Opossum (Node), Pybreaker, Gobreaker
5 enum CircuitState {
6     CLOSED = 'closed',
7     OPEN = 'open',
8     HALF_OPEN = 'half_open',
9 }
10
11 interface CircuitBreakerConfig {
12     failureThreshold: number; // Failures before opening
13     successThreshold: number; // Successes to close from half-open
14     timeout: number; // Time in open state before half-open
15 }
16
17 class CircuitBreaker {
18     private state: CircuitState = CircuitState.CLOSED;
19     private failures: number = 0;

```



```

20     private successes: number = 0;
21     private lastFailureTime: number = 0;
22     private openTime: number = 0;
23
24     constructor(
25         private name: string,
26         private config: CircuitBreakerConfig,
27     ) {}
28
29     async execute<T>(operation: () => Promise<T>, fallback?: () => T): Promise<T> {
30         if (this.state === CircuitState.OPEN) {
31             if (Date.now() - this.openTime > this.config.timeout) {
32                 this.state = CircuitState.HALF_OPEN;
33                 this.successes = 0;
34             } else {
35                 // Circuit is open, fail fast
36                 if (fallback) return fallback();
37                 throw new Error(`Circuit ${this.name} is open`);
38             }
39         }
40
41         try {
42             const result = await operation();
43             this.onSuccess();
44             return result;
45         } catch (error) {
46             this.onFailure();
47             if (fallback) return fallback();
48             throw error;
49         }
50     }
51
52     private onSuccess(): void {
53         this.failures = 0;
54         if (this.state === CircuitState.HALF_OPEN) {
55             this.successes++;
56             if (this.successes >= this.config.successThreshold) {
57                 this.state = CircuitState.CLOSED;
58             }
59         }
60     }
61
62     private onFailure(): void {

```



```
63     this.failures++;
64     this.lastFailureTime = Date.now();
65
66     if (this.failures >= this.config.failureThreshold) {
67         this.state = CircuitState.OPEN;
68         this.openTime = Date.now();
69     }
70 }
71 }
```

The fallback is critical. A circuit breaker that just throws errors is only marginally better than no circuit breaker. Good fallbacks include: returning cached data, returning a degraded response, or returning a static default. The user experience should be “this feature is temporarily unavailable” rather than “the entire site is down.”

Bulkheads for Isolation

The bulkhead pattern comes from ship design. A ship’s hull is divided into watertight compartments. If one compartment floods, the bulkheads contain the damage – the ship stays afloat because the other compartments are isolated.

In software, bulkheads isolate different workloads into separate resource pools. If your analytics queries suddenly go rogue and consume all available database connections, that shouldn’t affect your payment processing. Each workload gets its own pool, and exhausting one pool doesn’t starve the others.

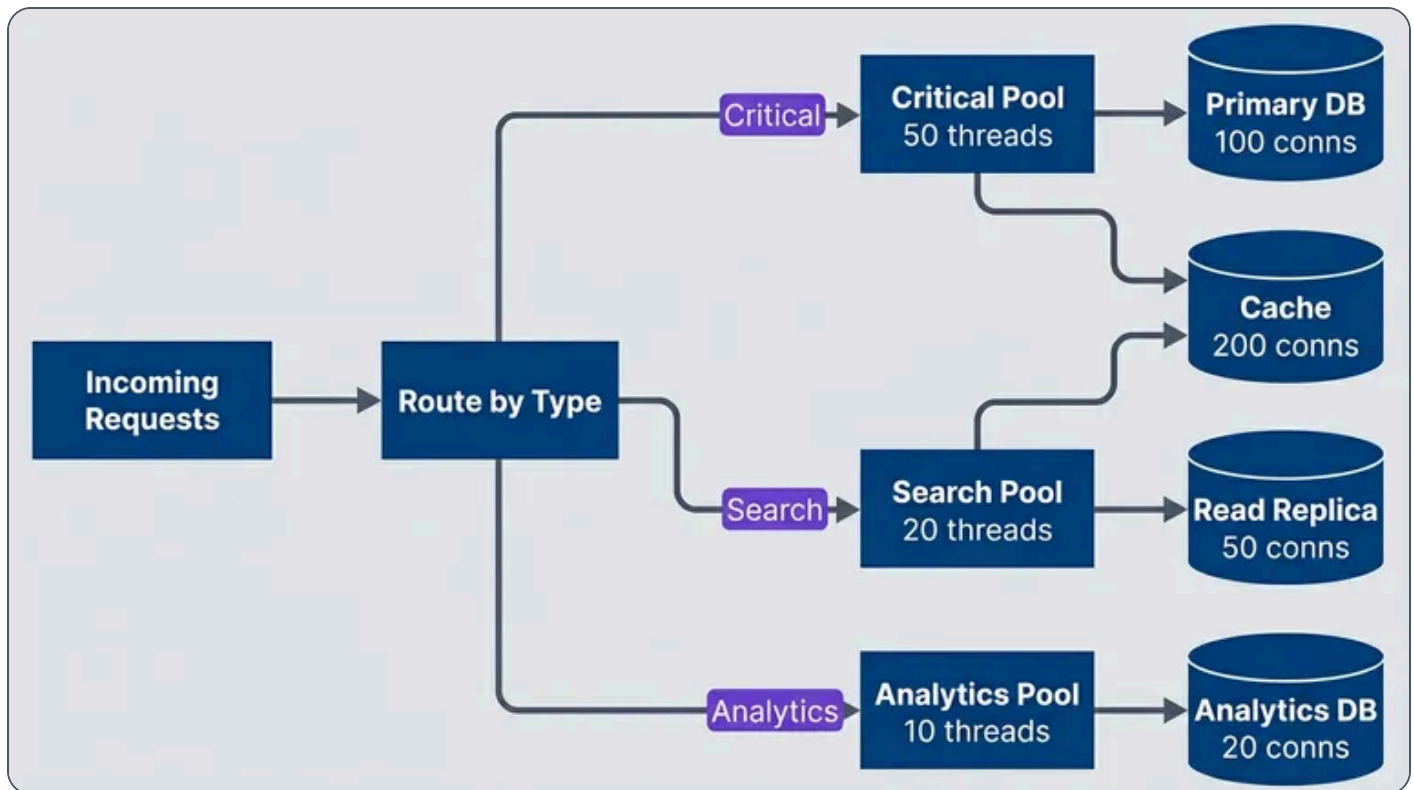
```
1  # Spring Boot application.yml with Resilience4j bulkhead configuration.
2  # Thread pool bulkheads limit concurrent calls per operation type.
3  resilience4j:
4    bulkhead:
5      instances:
6        criticalApi:
7          maxConcurrentCalls: 50
8          maxWaitDuration: 100ms
9        search:
10         maxConcurrentCalls: 20
11         maxWaitDuration: 500ms
12        analytics:
13         maxConcurrentCalls: 10
```



```
14         maxWaitDuration: 5s
15
16     thread-pool-bulkhead:
17         instances:
18             criticalApi:
19                 maxThreadPoolSize: 50
20                 coreThreadPoolSize: 25
21                 queueCapacity: 100
22             search:
23                 maxThreadPoolSize: 20
24                 coreThreadPoolSize: 10
25                 queueCapacity: 50
26             analytics:
27                 maxThreadPoolSize: 10
28                 coreThreadPoolSize: 5
29                 queueCapacity: 20
30
31     # HikariCP connection pools - separate pools per workload
32     spring:
33         datasource:
34             primary:
35                 hikari:
36                     maximum-pool-size: 100
37                     minimum-idle: 20
38                     connection-timeout: 1000
39             read-replica:
40                 hikari:
41                     maximum-pool-size: 50
42                     minimum-idle: 10
43                     connection-timeout: 500
```

The sizing reflects priority. Critical APIs get 50 threads and fast timeouts – they need to respond quickly or not at all. Analytics gets only 10 threads and longer timeouts – it’s acceptable for reports to take 30 seconds, but it’s *not* acceptable for analytics to consume resources needed elsewhere.





Bulkhead architecture isolating different workloads into separate resource pools

✓ SUCCESS

Bulkheads ensure that a slow analytics query cannot exhaust the connection pool needed for critical payment processing. Isolation prevents cascading failures across unrelated functionality.

Adaptive Concurrency Limits

Static concurrency limits are a guess. You pick a number, deploy it, and hope it's right. If your dependencies get faster (new hardware, query optimization), you're leaving capacity on the table. If they get slower (more data, degraded network), you're overloading them.

Adaptive concurrency limits adjust automatically based on observed behavior. The algorithm uses AIMD (Additive Increase Multiplicative Decrease) – Additive Increase Multiplicative Decrease – the same approach TCP uses for congestion control. When latency is healthy, slowly increase the limit. When latency exceeds



target, cut the limit significantly. This finds the optimal concurrency for current conditions without manual tuning.

adaptive-concurrency.ts

```

1  // Use in app code
2  interface AdaptiveLimiterConfig {
3    initialLimit: number;
4    minLimit: number;
5    maxLimit: number;
6    targetLatency: number;    // Target P99 latency
7    smoothing: number;        // 0-1, how fast to adjust
8  }
9
10 class AdaptiveConcurrencyLimiter {
11   private limit: number;
12   private inFlight: number = 0;
13   private latencies: number[] = [];
14
15   constructor(private config: AdaptiveLimiterConfig) {
16     this.limit = config.initialLimit;
17   }
18
19   async acquire(): Promise<boolean> {
20     if (this.inFlight >= this.limit) {
21       return false;
22     }
23     this.inFlight++;
24     return true;
25   }
26
27   release(latencyMs: number): void {
28     this.inFlight--;
29     this.latencies.push(latencyMs);
30
31     // Adjust limit periodically
32     if (this.latencies.length >= 100) {
33       this.adjustLimit();
34       this.latencies = [];
35     }
36   }
37

```




```
38     private adjustLimit(): void {
39         const p99 = this.calculateP99();
40
41         if (p99 < this.config.targetLatency * 0.8) {
42             // Latency well below target, increase limit (AIMD: additive increase)
43             this.limit = Math.min(
44                 this.config.maxLimit,
45                 this.limit + 1
46             );
47         } else if (p99 > this.config.targetLatency) {
48             // Latency above target, decrease limit (AIMD: multiplicative decrease)
49             this.limit = Math.max(
50                 this.config.minLimit,
51                 Math.floor(this.limit * 0.9)
52             );
53         }
54     }
55
56     private calculateP99(): number {
57         const sorted = [...this.latencies].sort((a, b) => a - b);
58         const index = Math.floor(sorted.length * 0.99);
59         return sorted[index];
60     }
61 }
```

Adaptive concurrency limiter using AIMD (Additive Increase Multiplicative Decrease) (Additive Increase Multiplicative Decrease).

If you'd rather not implement this in application code, infrastructure-level options exist. Envoy's adaptive concurrency filter applies the same AIMD (Additive Increase Multiplicative Decrease) algorithm at the proxy layer – useful in service mesh deployments where you want consistent behavior across polyglot services without code changes. AWS ALB offers some adaptive behavior through its “least outstanding requests” routing algorithm, which naturally sends traffic to less-loaded targets, but it's coarser-grained: ALB doesn't implement true AIMD (Additive Increase Multiplicative Decrease) or expose the concurrency limit for tuning, so it won't protect individual downstream services the way application-level or sidecar-level limiters can.

The asymmetry in AIMD (Additive Increase Multiplicative Decrease) is intentional. Additive increase (+1) means slow, cautious growth when things are good. Multiplicative decrease ($\times 0.9$) means rapid reduction when things are bad. This responds quickly to overload while avoiding wild oscillations during normal operation. Netflix's



`concurrency-limits` library implements this pattern with additional sophistication – worth considering before rolling your own.

Testing Overload Handling

Load Testing for Backpressure

Building backpressure mechanisms isn't enough – you need to verify they work. Standard load tests measure throughput and latency under expected conditions. Backpressure testing deliberately pushes *beyond* capacity to validate graceful degradation.

The key insight: you're not testing whether your system can handle the load. You're testing whether it *fails correctly* when it can't. A successful test might show 50% of requests rejected with 503s – that's the system protecting itself, not a failure.

INFO

Load testing tools like Grafana's k6 <<https://grafana.com/docs/k6/latest/get-started/>> and Locust <<https://docs.locust.io/en/stable/what-is-locust.html>> work well for these scenarios.

I've found three scenarios cover most backpressure validation needs:

load-test-scenarios.yaml

```
1  scenarios:
2    - name: "Gradual ramp to saturation"
3      description: "Verify graceful degradation as load increases"
4      phases:
5        - duration: 2m
6          rate: 100rps # Below capacity
7        - duration: 2m
8          rate: 200rps # At capacity
9        - duration: 2m
10         rate: 400rps # Above capacity
11        - duration: 2m
```



```
12         rate: 100rps # Recovery
13     assertions:
14     - "p99_latency < 1s during all phases"
15     - "error_rate < 5% when rate <= capacity"
16     - "503_rate increases gracefully above capacity"
17     - "recovery to normal within 30s of load reduction"
18
19 - name: "Sudden spike"
20   description: "Verify system survives traffic spike"
21   phases:
22   - duration: 1m
23     rate: 100rps # Baseline
24   - duration: 30s
25     rate: 1000rps # 10x spike
26   - duration: 2m
27     rate: 100rps # Return to baseline
28   assertions:
29   - "no complete outage during spike"
30   - "goodput remains > 50% of baseline during spike"
31   - "full recovery within 60s after spike"
32
33 - name: "Sustained overload"
34   description: "Verify stable degradation under prolonged overload"
35   phases:
36   - duration: 10m
37     rate: 300rps # 50% above capacity
38   assertions:
39   - "system remains responsive throughout"
40   - "no resource exhaustion (memory, connections)"
41   - "error rate stabilizes (not increasing)"
```

Load test scenarios specifically designed to validate backpressure mechanisms.

The “gradual ramp” scenario is the foundation. Watch for these signals as you cross capacity: queue depths should grow but plateau (shedding kicks in), 503 rates should increase proportionally (admission control working), and latency for *successful* requests should stay bounded (CoDel (Controlled Delay) or similar doing its job). The recovery phase is equally important – a system that can’t return to normal after overload subsides has a leak somewhere.



The “sudden spike” scenario simulates flash crowds – Black Friday launches, viral posts, or just someone’s misconfigured retry loop. The goal isn’t zero errors during the spike; it’s *survival*. If goodput drops to zero during a 10x spike, your system is too brittle. If it maintains 50% of baseline throughput while rejecting the excess, that’s success.

“Sustained overload” catches a different class of bugs: resource leaks under pressure. A system might handle a 2-minute spike just fine but accumulate dead connections, grow heap indefinitely, or exhaust file descriptors over 10 minutes of steady overload. The key assertion is *stabilization*—error rates should plateau, not keep climbing.

⚠ WARNING

The key is the assertions – don’t just measure throughput, explicitly verify that backpressure mechanisms activated at the right thresholds.

Chaos Engineering for Overload

Load testing validates behavior under high request volume. Chaos engineering validates behavior under *unexpected failures*—the scenarios that trigger backpressure mechanisms in ways traffic alone can’t simulate.

Experiment	Method	Expected Behavior
● Queue bomb	Fill queues to capacity	System sheds load, stays responsive
● Thread starvation	Block worker threads	Circuit breakers trip, fallbacks activate
● Memory pressure	Allocate memory in dependency	GC pressure handled, graceful degradation
● Connection exhaustion	Hold connections open	New requests rejected cleanly
● Cascade trigger	Fail upstream service	Downstream services shed load appropriately

Chaos experiments for validating overload handling.



Connection exhaustion is my go-to first chaos experiment because it's both common in production and easy to simulate. Database connection pools are the usual culprit – a slow query holds a connection, the pool fills, and suddenly every request is waiting for a connection that won't come.

```
1  #!/bin/bash
2  # Chaos experiment: Verify system survives connection exhaustion
3
4  echo "Starting connection exhaustion experiment"
5
6  # Hold 90% of database connections
7  for i in {1..90}; do
8      psql -h db.example.com -c "SELECT pg_sleep(300)" &
9  done
10
11 echo "Holding 90 connections for 5 minutes"
12 echo "Monitoring system behavior..."
13
14 # Monitor during experiment
15 for i in {1..60}; do
16     curl -s http://app.example.com/health | jq '.status'
17     curl -s http://app.example.com/metrics | grep 'http_requests_total'
18     sleep 5
19 done
20
21 # Cleanup
22 echo "Releasing connections"
23 pkill -f "pg_sleep"
24
25 echo "Experiment complete. Check dashboards for behavior analysis."
```

What should happen when you run this? If your bulkheads are working, the connection-starved workload should degrade while others continue normally. If your circuit breakers are working, requests that can't get connections should fail fast rather than timeout. If neither is working, you'll see cascading failure – the connection wait blocks threads, which blocks request processing, which backs up the request queue, which exhausts memory. That's exactly what you're testing for.

For thread starvation, inject artificial latency into a dependency (a chaos proxy like Toxiproxy works well). Watch for circuit breakers tripping and fallbacks activating. For memory pressure, use stress-ng or similar to consume memory on the host running a dependency. Watch for graceful degradation rather than OOM kills propagating to callers.



 DANGER

Never run chaos experiments in production without proper safeguards. Start in staging, have kill switches ready, and ensure the blast radius is contained.

Testing tells you whether your backpressure mechanisms work. But in production, you need to *see* them working – or not working – in real time. That’s where observability comes in.

Observability for Overload

Key Metrics to Monitor

You can’t manage what you can’t measure – and during overload, you need to measure the *right* things. Standard throughput and latency metrics tell you how the system is performing, but backpressure observability requires metrics that tell you *why* and *where* the system is protecting itself.

I organize overload metrics into three categories: capacity indicators (how close are we to limits?), backpressure indicators (are protection mechanisms activating?), and health indicators (are we still serving users effectively?).

```
1  # Pseudocode
2  metrics:
3    # Capacity indicators
4    - name: request_rate
5      type: counter
6      labels: [endpoint, status]
7      alert: "rate > capacity_threshold"
8
9    - name: concurrent_requests
10     type: gauge
11     labels: [service]
12     alert: "value > concurrency_limit * 0.9"
13
14   - name: queue_depth
15     type: gauge
```



```

16     labels: [queue_name]
17     alert: "value > max_depth * 0.8"
18
19 # Backpressure indicators
20 - name: requests_rejected
21   type: counter
22   labels: [reason] # rate_limit, queue_full, circuit_open
23
24 - name: shed_requests
25   type: counter
26   labels: [tier, reason]
27
28 - name: retry_after_seconds
29   type: histogram
30   labels: [endpoint]
31
32 # Health indicators
33 - name: goodput_ratio
34   type: gauge
35   description: "successful_requests / total_requests"
36   alert: "value < 0.9"
37
38 - name: latency_p99
39   type: histogram
40   labels: [endpoint]
41   alert: "value > slo_threshold"

```

Those metrics matter because they tell you three different things: how close you are to overload, whether your protection mechanisms are firing, and whether users are still getting successful responses.

➤ Capacity indicators:

Your early warning system. When `concurrent_requests` crosses 90% of your concurrency limit, you're not in trouble yet – but you're one traffic spike away. When `queue_depth` grows faster than it drains, backpressure is about to kick in. These metrics let you see overload coming before users notice.

➤ Backpressure indicators:

Tells you when protection mechanisms are active. A spike in `requests_rejected` with reason `rate_limit` means your rate limiter is doing its job. If you see rejections with reason `circuit_open`, a downstream dependency is struggling. The `shed_requests` counter broken down by tier shows which degradation level you're operating at. These metrics answer the question: "Is the system protecting itself correctly?"



➤ **Health indicators:**

The bottom line. `goodput_ratio` is the most important – it's the percentage of requests that actually succeeded. A 503 is a successful rejection (the system protected itself), but it's not goodput. If goodput drops below your SLO, users are suffering regardless of how well your backpressure mechanisms work. Latency percentiles matter too: p99 staying bounded while p50 degrades slightly is healthy backpressure; p99 exploding means something's queuing that shouldn't be.

Of those three categories, the rejection metrics are usually the fastest way to identify which protective layer is currently absorbing the pressure.

INFO

The `requests_rejected` label reason is critical. During an incident, you need to know *which* mechanism is rejecting – rate limiter, circuit breaker, queue full, or admission control. Different reasons point to different root causes.

Dashboard Layout

A good overload dashboard tells a story at a glance. When you're paged at 3 AM, you shouldn't have to hunt through 15 panels to understand what's happening. I've found a four-section layout works well:

#	Section	Panels	Purpose
1	Load & Capacity	Request Rate, Concurrency, Queue Depth	Current vs. max capacity indicators
2	Backpressure Activity	429s/min, 503s/min, Shed Rate, Circuit State	Active backpressure mechanisms
3	Health Indicators	Goodput %, P99 Latency, Error Rate	Are we serving users effectively?
4	Degradation State	Current Tier, Features Disabled	Operational mode at a glance

Dashboard sections and their panels for overload monitoring.



That layout works because each section answers a different on-call question, and the order mirrors how you triage an overload event in real time.

1

Load & Capacity:

Goes at the top because it's your first question: "How much traffic are we getting, and how close are we to limits?" Show request rate as a line with capacity threshold as a horizontal marker. Show concurrency and queue depth as gauges with red zones. At a glance, you should see whether you're at 50% capacity or 95%.

2

Backpressure Activity:

Answers the second question: "Is the system protecting itself?" Show rejection rates by type—429s (rate limiting), 503s (admission control), circuit breaker state (open/closed/half-open). If these counters are at zero, you're not in overload. If they're spiking, the system is actively shedding load. This section turns from green to yellow to red as backpressure intensifies.

3

Health Indicators:

Answers the critical question: "Are users being served?" Goodput percentage should be front and center – if it's above 95%, things are okay even if you're shedding some load. P99 latency shows whether successful requests are still fast. Error rate distinguishes between deliberate rejections (503s you sent) and unexpected failures (5xxs from bugs).

4

Degradation State:

Shows operational mode at a glance. If you've implemented graceful degradation tiers, show the current tier prominently – a big "ELEVATED" or "CRITICAL" badge tells the on-call engineer immediately that the system is in a degraded state. List which features are disabled so they don't waste time investigating why search isn't working.

When those sections are arranged well, the dashboard stops being a wall of charts and starts reading like a short incident narrative.



✓ SUCCESS

During an incident, read the dashboard top to bottom: “We’re at 120% capacity (Load), rejecting 20% of requests (Backpressure), maintaining 85% goodput (Health), in ELEVATED mode with recommendations disabled (Degradation).” That’s a complete situational picture in 10 seconds.

Conclusion

Every system has limits. The question isn’t whether yours will face overload — it’s whether it will handle overload gracefully or collapse catastrophically. The patterns in this article share a common philosophy: *admit your limits, communicate them clearly, and degrade predictably.*

Here’s what that looks like in practice:

- ✓ **Fast rejection beats slow failure:** —When you can’t serve a request, say so immediately. A 503 returned in 5ms is infinitely better than a timeout after 30 seconds. The fast rejection frees resources, gives the client useful information, and preserves capacity for requests you *can* serve. Every slow failure is a cascading failure waiting to happen.
- ✓ **Backpressure requires cooperation:** —Your server-side mechanisms — rate limiting, admission control, circuit breakers — only work if clients understand and respect the signals. Return proper status codes (429, 503). Include `Retry-After` headers. Document your backpressure behavior. And if you control the clients, implement exponential backoff with jitter. A system where servers shed load but clients immediately retry is a system that’s fighting itself.
- ✓ **Isolation contains failures:** —Bulkheads ensure that a runaway analytics query can’t starve your payment processing. Circuit breakers ensure that a failing dependency can’t take down callers. Separate resource pools, separate failure domains. When something breaks — and something always breaks — the blast radius should be contained.
- ✓ **Graceful degradation preserves what matters:** —Not all features are equally important. During overload, shed the non-essential (personalization, recommendations, fancy animations) to preserve the critical (authentication, checkout, core functionality). Work with product owners to classify features *before* the incident. The middle of an outage is the wrong time to debate priorities.
- ✓ **Observability makes it all work:** —You can’t manage what you can’t measure. Track capacity indicators to see overload coming. Track backpressure indicators to verify protection mechanisms are activating. Track health indicators to know if users are actually being served. Build dashboards that tell the story at a glance.



The underlying principle is simple: *some* work completing successfully is better than *all* work failing. A system that accepts 2,000 requests per second when it can only handle 1,000 will serve nobody well. A system that accepts 1,000 and rejects 1,000 with clear signals serves half its users perfectly and gives the other half actionable information.

Backpressure isn't about handling unlimited load — nothing can do that. It's about maintaining service quality within your design capacity and degrading predictably beyond it. Build systems that know their limits, and you'll build systems that survive.

Copyright © 2022 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/backpressure-load-shedding-admission-control-overload>

